



—

# Python Fundamentals

*Viktor Vazharski, DevOps Consultant  
Regional School of DevOps 2022*

# AGENDA

---

- Introduction
- Python environments
- Variables and Data Types
- Objects and Data Structures
- Operators and Operands
- Homework

### What is Python?

- Python is high-level scripting language
  - Implements the imperative
  - Object-orientated
  - Functional paradigms
- Open source and communitè driven
- Source can be compiled or run just-in-time

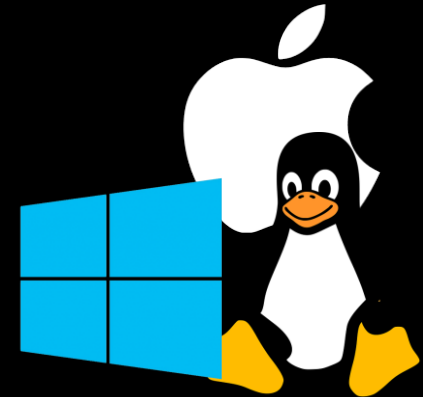


# Introduction

---

## Why Python?

- No universal agreement on which scripting language is the best
- One of the most flexible, reliable and efficient programming language
- Easy to use and learn
- Hundreds of Python Libraries and Frameworks
- Python applications are universal



# Introduction

---

## Versions

- Current production versions are 2.x.x and 3.x.x
- Both version are stable and suitable for use
- Python3: relatively major redesign
  - Not backward compatible



# Introduction

---

## Interesting features

- White space has semantics
- Statements are terminated by <Enter>
- No variable declaration
- Dynamic typing
- Associative arrays

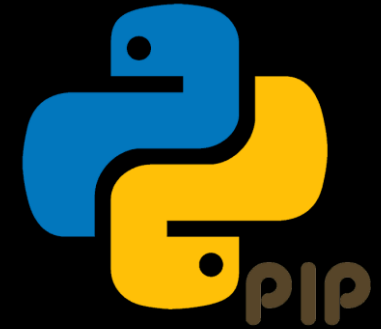


# Introduction

---

## PIP & Requirements file

- Package manager
- Install and manage additional libraries and dependencies
- Included in Python installers
- requirements.txt
- Fine-Tuning Requirements
- Freezing Requirements per environment

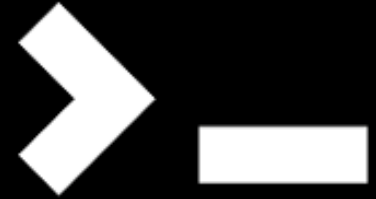


# Python environments

---

## Development environments

- IDEs - cross-platform development environments:
  - PyCharm
  - Visual Studio Code
  - Etc.
- Python Shell



# Python environments

---

## Virtual Environments

- Pipenv:
  - Packaging tool
  - Simplifies the development process
- Virtual environments
  - Isolated environment for projects
- Docker



# Variables & Data Types

---



## Python Elements

- Keywords
  - Reserved words
- Identifiers
  - Names given to entities
  - Writing rules
- Statements
  - Single line statement
  - Multiline statement

```
#Single line statement  
print("Hello World!")  
#Multiline statement  
print("Hello "); print("World!")
```

## Variables & Data Types

---

### Explicit line continuation

```
string = "This" + " is" + " a" + " string" \  
+ " with" + " a" + " double" + " line" + " value"  
  
print(string)
```

### Implicit line continuation

```
string = ("This" + " is" + " a" + " string"  
+ " with" + " a" + " double" + " line" + " value")  
  
print(string)
```

# Variables & Data Types

---

## Keywords

|                       |                      |                       |                     |
|-----------------------|----------------------|-----------------------|---------------------|
| <code>False</code>    | <code>def</code>     | <code>if</code>       | <code>raise</code>  |
| <code>None</code>     | <code>del</code>     | <code>import</code>   | <code>return</code> |
| <code>True</code>     | <code>elif</code>    | <code>in</code>       | <code>try</code>    |
| <code>and</code>      | <code>else</code>    | <code>is</code>       | <code>while</code>  |
| <code>as</code>       | <code>except</code>  | <code>lambda</code>   | <code>with</code>   |
| <code>assert</code>   | <code>finally</code> | <code>nonlocal</code> | <code>yield</code>  |
| <code>break</code>    | <code>for</code>     | <code>not</code>      |                     |
| <code>class</code>    | <code>from</code>    | <code>or</code>       |                     |
| <code>continue</code> | <code>global</code>  | <code>pass</code>     |                     |

# Variables & Data Types

---

## Variables

- Automatically declared
- Variables contains references to data values
- Created when value is assigned
- Don't use a variable name in a statement until has been assigned a value
- No static types



# Variables & Data Types

---



## Data Types

- Numeric Types:
  - Integer
    - Sequence of decimal digits without any prefix to be a decimal number
    - Binary, octal and hexadecimal values
  - Floats
    - Values specified with a decimal point
  - Complex
    - Specified as <real part>+<imaginary part>j
- Boolean
  - True or False
- Strings
  - Sequences of character data

# Variables & Data Types

---



## Type Conversion Functions

- Implicit Type Conversion
  - Interpreter automatically converts one data type to another
- Explicit Type Conversion
  - Data type is manually changed by the user as per their requirement

# Variables & Data Types

---

## Implicit Type Conversion

```
x = 10
print("x is of type:", type(x))

y = 10.6
print("y is of type:", type(y))

x = x + y
print(x)
print("x is of type:", type(x))
```

```
x is of type: <class 'int'>
y is of type: <class 'float'>
20.6
x is of type: <class 'float'>
```

# Variables & Data Types

---

## Explicit Type Conversion

```
s = "10010"

a = int(s)
print ("After converting to integer : ", end="")
print (a)

c = int(s,2)
# == 1*(2**2) + 0*(2**1) + 0*(2**1) + 1*(2**0) + 0*(2**1) == 18
print ("After converting to integer base 2 : ", end="")
print (c)

e = float(s)
print ("After converting to float : ", end="")
print (e)
```

```
After converting to integer : 10010
After converting to integer base 2 : 18
After converting to float : 10010.0
```

# Objects & Data Structures

---

## Objects

- Object-oriented programming (OOP) concept
- Objects bundle properties and behaviors

```
>>> print(300)
```



# Objects & Data Structures

---

Variable is a named location used to store data in memory

>>> n = 7



>>> m = n



# Objects & Data Structures

---

## Data Structures

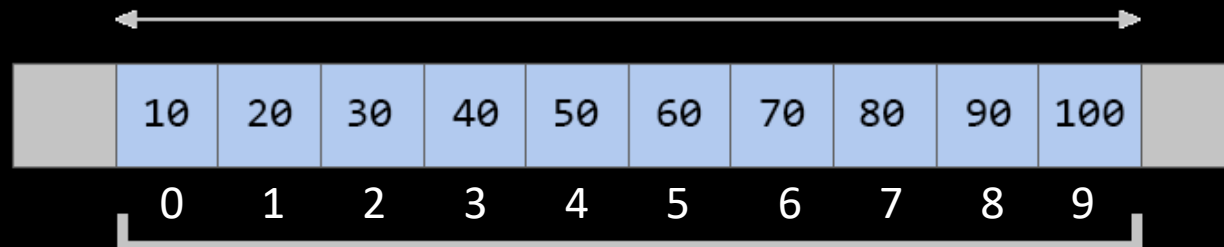
- Array Data Structures:
  - List
  - Tuple
- Dictionaries, Maps and Hash Tables:
  - Dict
- Sets

# Objects & Data Structures

---

## Array Data Structures

- Fixed-size data record that allow each element to be efficiently located based on its index
- Contiguous data structures



# Objects & Data Structures

---

## Lists

[ ]

- Comma-separated sequence of items, enclosed in square brackets

```
>>> myList = ["one", "two", "three"]
```
- Can contain heterogeneous data

```
>>> myList = ["one", "two", "3"]
```
- Mutable

```
>>> myList = "hello"  
['one', 'hello', 3]
```
- Can be sliced, indexed, concatenated and repeated

# Objects & Data Structures

---

## Tuples

( )

- Like a list, tuples are iterable arrays of objects
- Immutable - unchangable
- Item separated by commas are enclosed within brackets ( )
- Example uses of tuples:
  - Country Names
  - Ordered set of functions
  - Land Use Codes

```
t = (1, 2, 'Python', (14, 'Viktor'))
```

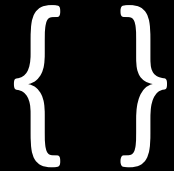
Diagram illustrating tuple indexing for the tuple `t`:

- `t[0]` points to the first element `1`.
- `t[1]` points to the second element `2`.
- `t[3]` points to the third element `'Python'`.
- `t[4]` points to the fourth element, which is a nested tuple `(14, 'Viktor')`.

# Objects & Data Structures

---

## Dictionaries



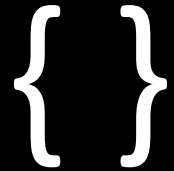
- Ordered collection of elements
- Mutable
- Indexed by keys
- Consist of <key, value> pairs
- Keys are the unique identifiers
- Dict is defined within curly brackets {}

```
myDict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964,  
    "colors": ["red", "white", "blue"]  
}
```

# Objects & Data Structures

---

## Sets



- Unordered collection of objects
- Doesn't allow duplicate elements
- Mutable
- Backed by the dict data type and share the same performance characteristics
- Use cases:
  - Test a value for membership in a set
  - Insert or delete new values from a set
  - Compute the union or intersection of two sets

## Operators & Operands

---

- Operators are special symbols that carry out arithmetic or logical computation
- The value that operator operates on is called the operand
- Example:
  - >>> 2 + 3
  - 5
  - + is operator that performs addition
  - 2 and 3 are the operands and 5 is the output of the operation

# Operators & Operands

---

## Arithmetic Operators

- Used to perform mathematical operations
  - Addition  
 $x + y$
  - Subtraction  
 $x - y$
  - Multiplication  
 $x * y$
  - Division  
 $x / y$
  - Modulus  
 $x \% y$
  - Exponentiation  
 $x ** y$
  - Floor division  
 $x // y$

# Operators & Operands

---

## Assignment Operators

- Used to perform operations on values and variables
  - Carry out arithmetic, logical, bitwise computations
- 
- |                       |                            |
|-----------------------|----------------------------|
| ○ Assign              | ○ Divide AND Assign        |
| $x = y + z$           | $a /= b$                   |
| ○ Add and Assign      | ○ Modulus AND Assign       |
| $a += b$              | $a %= b$                   |
| ○ Subtract AND Assign | ○ Divide(floor) AND Assign |
| $a -= b$              | $a //= b$                  |
| ○ Multiply AND Assign | ○ Exponent AND Assign      |
| $a *= b$              | $a **= b$                  |

# Operators & Operands

---

## Assignment Operators

- Performs Bitwise AND and Assign  
a &= b
- Performs Bitwise OR and Assign  
a |= b
- Performs Bitwise XOR and Assign  
a ^= b
- Performs Bitwise right shift and Assign  
a >>= b
- Performs Bitwise left shift and Assign  
a <<= b

# Operators & Operands

---

## Comparison Operators

- Also called relational operators
- Compare the values of two operands and return True or False
  - Equal  
 $a == b$
  - Not equal  
 $a != b$
  - Greater than  
 $a > b$
  - Less than  
 $a < b$
  - Greater than or equal to  
 $a >= b$
  - Less than or equal to  
 $a <= b$

# Operators & Operands

---

## Logical Operators

- Used for conditional statements

### AND operator

- returns TRUE if both the operands (right side and left side) are true

### OR operator

- returns TRUE if either of the operand (right side or left side) is true

### NOT operator

- returns TRUE if operand is false

# Iterators

---

## Looping with For

- Definite iteration
- Allows to loop over a block of code a set number of times
- Used for manipulating data structures

```
fruits = ['apple', 'banana', 'strawberry']  
for i in fruits:  
    print(i)
```